

RENEWABLY.IE

Postmark API Integration Reference

Developer Technical Documentation

Version 1.0 | April 2026

This document provides a comprehensive reference for integrating and working with the Postmark email service within the Renewably CRM platform. It covers API authentication, email sending patterns, webhook configuration, error handling, tag conventions, and the internal helper functions used across the application.

Table of Contents

1. Overview and Architecture
2. Authentication
3. Core API Functions
4. Specialised Email Functions
5. Postmark Tag Conventions
6. Email Templates and HTML Structure
7. Error Handling and Retry Logic
8. Webhook Configuration
9. Environment Variables
10. Testing and Debugging
11. Rate Limits and Quotas
12. CRM Module Integration Map
13. Database Integration (email_log Table)
14. Migration from Dev to Production
15. Quick Reference Card

1. Overview and Architecture

The Renewably CRM uses the Postmark email delivery service for all transactional email communication. Postmark was selected for its high deliverability rates (99.5%+ inbox placement), fast API response times, robust webhook infrastructure, and excellent developer experience. The integration is built as a lightweight module in `src/lib/postmark.ts` that wraps the Postmark REST API using native JavaScript `fetch()`, avoiding any third-party SDK dependency. This design choice reduces bundle size, eliminates versioning conflicts, and gives the team full control over request formatting and error handling logic.

All emails are sent from the verified sender address `hello@renewably.ie`, which must be configured as a verified sending domain or sender signature in the Postmark dashboard. The module supports two primary sending patterns: raw HTML emails (for custom-branded transactional messages) and Postmark template emails (for standardised communications managed through the Postmark UI). Every email is tagged with a descriptive identifier for tracking, analytics, and filtering within the Postmark activity dashboard.

The email system integrates with multiple CRM modules including the public contact form, the AI chatbot lead capture flow, proposal delivery, invoice notification, and the meeting scheduler. Each integration point uses a specialised wrapper function that handles HTML template rendering, recipient formatting, and tag assignment automatically. This ensures consistent branding, proper XSS escaping of user-submitted content, and reliable error logging across all email touchpoints.

1.1 Key Design Decisions

Decision	Rationale	Impact
No Postmark SDK	Native <code>fetch()</code> reduces bundle size and eliminates dependency versioning issues	Smaller deployment, full control over API calls
Branded HTML templates	Inline CSS email templates with Renewably branding (dark header, yellow accent)	Consistent brand identity across all customer touchpoints
Tag-based tracking	Every email tagged with a semantic identifier (e.g. "contact-form", "proposal-sent")	Easy filtering in Postmark dashboard and analytics
Graceful degradation	If <code>POSTMARK_SERVER_TOKEN</code> is not set, emails are silently skipped with a console warning	Development works without Postmark credentials
XSS escaping	All user-submitted content is escaped via <code>escapeHtml()</code> before embedding in HTML templates	Prevents XSS in email clients that render HTML

Table 1: Key design decisions in the Postmark integration

2. Authentication

Postmark uses a server-level API token for authentication. Every API request must include the `X-Postmark-Server-Token` HTTP header with the server token value. This token is associated with a specific Postmark server (essentially a sending domain or subdomain) and controls which sending

addresses, templates, and webhook configurations are available. The token is retrieved from the POSTMARK_SERVER_TOKEN environment variable at runtime.

The authentication system implements a graceful degradation pattern. If the environment variable is not set (common during local development), the module logs a warning and returns a synthetic success response with ErrorCode 0, allowing the rest of the application to continue functioning without crashing. This means developers can work on the full application stack locally without needing active Postmark credentials, which is particularly useful for frontend development and integration testing scenarios.

2.1 API Request Flow

```
POST https://api.postmarkapp.com/email/withTemplate<br/>Content-Type:
application/json<br/>Accept: application/json<br/>X-Postmark-Server-Token:
YOUR_SERVER_TOKEN_HERE<br/><br/>{<br/>  "From": "hello@renewably.ie",<br/>  "To":
  "recipient@example.com",<br/>  "Subject": "Welcome to Renewably",<br/>  "HtmlBody":
  "<h1>Welcome!</h1>",<br/>  "Tag": "welcome-auto-reply",<br/>  "TrackOpens": true,<br/>
  "TrackLinks": "HtmlOnly"<br/>}
```

2.2 Token Configuration

Environment	Variable	Source	Notes
Production	POSTMARK_SERVER_TOKEN	Postmark Dashboard > Servers > API Tokens	Create a dedicated token per environment
Staging	POSTMARK_SERVER_TOKEN	Separate Postmark server or same server	Use a different server to isolate test emails
Development	POSTMARK_SERVER_TOKEN	Not required	System degrades gracefully if unset

Table 2: Token configuration by environment

Security Note: Never commit POSTMARK_SERVER_TOKEN to version control. Store it in your .env.local file (locally) and in your hosting platform environment variables (production). Rotate tokens periodically and use the Postmark dashboard to monitor token usage and revoke compromised tokens immediately.

3. Core API Functions

The Postmark module exposes two core functions that map directly to the Postmark REST API endpoints. These functions handle all the low-level details of API communication including header construction, JSON serialisation, response parsing, and error handling. Higher-level specialised functions (covered in Section 4) build on top of these core functions to provide domain-specific email sending capabilities for the CRM.

3.1 sendEmail(options)

Sends a raw HTML email via the Postmark API. This is the primary function for sending custom-branded transactional emails. It accepts a SendEmailOptions object and returns a PostmarkResponse promise. The function automatically sets the sender address from the FROM_EMAIL environment variable (defaulting to

hello@renewably.ie), enables open tracking by default, and sets link tracking to "HtmlOnly" mode. Recipients can be specified as either a plain email string or an EmailRecipient object with name and email properties, which gets formatted into the standard "Name <email>" RFC 2822 format automatically.

Request Parameters

Parameter	Type	Required	Default	Description
to	string EmailRecipient	Yes	-	Primary recipient email address or object with name/email
cc	string EmailRecipient	No	-	Carbon copy recipient
bcc	string EmailRecipient	No	-	Blind carbon copy recipient
replyTo	string	No	-	Reply-to email address
subject	string	Yes	-	Email subject line
htmlBody	string	Yes	-	HTML content of the email
textBody	string	No	-	Plain text fallback for email clients without HTML support
tag	string	No	-	Postmark tag for analytics and filtering
trackOpens	boolean	No	true	Enable Postmark open tracking pixel
trackLinks	"None" "HtmlOnly" "All"	No	"HtmlOnly"	Link tracking mode

Table 3: sendEmail() parameters

Response Object (PostmarkResponse)

Field	Type	Description
ErrorCode	number	Postmark error code. 0 indicates success. Non-zero values indicate API errors.
Message	string	Human-readable status message from Postmark.
MessageID	string	Unique identifier for the sent message. Store this for tracking and support requests.
SubmittedAt	string	ISO 8601 timestamp of when the message was accepted by Postmark.
To	string	The recipient address the message was sent to.

Table 4: PostmarkResponse structure

Usage Example

```
import { sendEmail } from "@lib/postmark";  
  
const response = await  
sendEmail({  
  to: { email: "installer@company.ie", name: "John Murphy" },  
  subject: "Your Renewably account is ready",  
  htmlBody: "<h1>Welcome aboard!</h1><p>Your account has been set up.</p>",  
  textBody: "Welcome aboard! Your account has been set up.",  
  tag: "account-ready",  
});  
console.log("MessageID:", response.MessageID);
```

3.2 sendTemplate(options)

Sends an email using a Postmark template created in the Postmark dashboard. Template-based emails are ideal for standardised communications where the content structure is managed by the marketing or operations team through the Postmark UI, and the application only needs to supply dynamic data fields. This separation of concerns allows non-developers to modify email content without code changes, and ensures A/B testing and template versioning are handled by Postmark infrastructure.

The function requires a `TemplateId` (numeric, obtained from the Postmark template editor) and a `TemplateModel` object whose properties map to the template placeholder variables. Postmark supports both Handlebars and HTML template syntax. Open tracking is enabled by default. The function posts to the `/email/send` endpoint (note: different from `sendEmail` which uses `/email/withTemplate`), which is the dedicated template sending endpoint in the Postmark API.

Request Parameters

Parameter	Type	Required	Description
to	string EmailRecipient	Yes	Primary recipient
cc	string EmailRecipient	No	Carbon copy recipient
bcc	string EmailRecipient	No	Blind carbon copy recipient
replyTo	string	No	Reply-to address
templateId	number	Yes	Postmark template ID (from dashboard)
templateModel	Record<string, unknown>	Yes	Dynamic data for template placeholders
tag	string	No	Postmark tag for analytics
trackOpens	boolean	No	Enable open tracking (default: true)

Table 5: sendTemplate() parameters

Usage Example

```
import { sendTemplate } from "@lib/postmark";  
  
const response = await  
sendTemplate({  
  to: "installer@company.ie",  
  templateId: 3489201,  
  templateModel: {  
    installer_name: "John Murphy",  
    company_name: "SolarTech Ltd",  
    setup_url: "https://app.renewably.ie/onboarding/start",  
  },  
  tag: "onboarding-started",  
});
```

4. Specialised Email Functions

The CRM module provides four specialised email functions that handle specific business workflows. Each function manages its own HTML template rendering, recipient formatting, tag assignment, and subject line construction. These functions should be used instead of calling `sendEmail()` directly for their respective use cases, as they ensure consistent branding, proper escaping, and correct tagging across the platform.

4.1 `sendContactNotification(data)`

Sends a branded notification email to `hello@renewably.ie` whenever someone submits the public contact form on `renewably.ie`. This function is called from the POST `/api/contact` endpoint and includes all form fields: name, email, phone, company, message, source, and `jobsPerMonth`. The email is sent with Reply-To set to the enquirer's email address, allowing the team to respond directly from their email client. All user-submitted content is XSS-escaped before being embedded in the HTML template, preventing malicious content from executing in email clients.

Parameter	Type	Required	Description
<code>data.name</code>	string	Yes	Enquirer name
<code>data.email</code>	string	Yes	Enquirer email (also used as Reply-To)
<code>data.phone</code>	string	No	Phone number
<code>data.company</code>	string	No	Company name
<code>data.message</code>	string	Yes	Message content from the form
<code>data.source</code>	string	No	How they found Renewably
<code>data.jobsPerMonth</code>	string	No	Estimated monthly install volume

Table 6: `ContactNotificationData` parameters

Tag: "contact-form" | Recipient: `hello@renewably.ie` | Reply-To: enquirer email

4.2 `sendWelcomeEmail(name, email)`

Sends an automated welcome confirmation email to new contacts who submitted the contact form. This email serves as immediate acknowledgement that their message was received, sets expectations for the next steps in the sales process (discovery call, AI team deployment, approval), and provides direct contact details for urgent enquiries. The email template follows the Renewably brand guidelines with a dark header, yellow accent colour, and clean white content area. The phone number `+353 873958424` and email `hello@renewably.ie` are hardcoded in the template footer.

Tag: "welcome-auto-reply" | Recipient: the contact form submitter

4.3 `sendProposalEmail(data)`

Sends a proposal notification email to a customer. This is triggered when a proposal is created or sent from the CRM. The email includes the proposal title, total investment amount formatted in EUR (using Intl.NumberFormat with en-IE locale), validity period, and an optional link to view the full proposal online. The amount is displayed prominently in a highlighted card using the brand yellow (#F3D840) accent colour. If a proposalLink is provided, a call-to-action button is rendered in the email body. This function is called from the POST /api/crm/proposals/[id]/send endpoint.

Parameter	Type	Required	Description
data.proposalTitle	string	Yes	Title of the proposal
data.contactName	string	Yes	Customer name for personalisation
data.contactEmail	string	Yes	Customer email address
data.companyName	string	No	Customer company name
data.totalAmount	number	Yes	Total amount in EUR
data.validUntil	string	No	ISO date string for proposal expiry
data.proposalLink	string	No	URL to view the proposal online

Table 7: ProposalEmailData parameters

Tag: "proposal-sent" | Amount format: EUR via Intl.NumberFormat("en-IE")

4.4 sendInvoiceEmail(data)

Sends an invoice notification email to a customer. This function is triggered from the POST /api/crm/invoices/[id]/send endpoint when an invoice is marked as sent. The email displays the invoice number, amount due (formatted in EUR), and due date. If an invoiceLink is provided, a "View Invoice" call-to-action button is included. The email template mirrors the proposal email structure with a highlighted amount card and consistent brand styling. The subject line includes both the invoice number and amount for quick identification in the recipient inbox.

Tag: "invoice-sent" | Amount format: EUR via Intl.NumberFormat("en-IE")

5. Postmark Tag Conventions

Every email sent through the Postmark integration is tagged with a semantic identifier. Tags serve as the primary organisational tool within the Postmark activity dashboard, allowing the team to filter, search, and analyse email delivery by purpose. Tags are limited to 128 characters and should follow a consistent naming convention across the platform. The following table documents all currently defined tags, their associated sending functions, and the CRM modules that trigger them.

Tag	Function	Trigger Module	Description
-----	----------	----------------	-------------

contact-form	sendContactNotification()	POST /api/contact	New enquiry notification to the team
welcome-auto-reply	sendWelcomeEmail()	POST /api/contact	Auto-reply confirmation to enquirer
proposal-sent	sendProposalEmail()	POST /api/crm/proposals/[id]/send	Proposal delivered to customer
invoice-sent	sendInvoiceEmail()	POST /api/crm/invoices/[id]/send	Invoice delivered to customer
chat-lead-capture	sendContactNotification()	POST /api/chat	AI chat detected a buying signal
meeting-reminder	(future)	Workflow automation	Upcoming meeting reminder
onboarding-started	(future)	sendTemplate()	Installer onboarding initiated

Table 8: Postmark tag conventions and mappings

When adding new email types to the CRM, always define a new tag following the kebab-case naming pattern shown above. Avoid generic tags like "email" or "notification" as they provide no analytical value. The tag should describe the business action that triggered the email, not the email content. For example, use "proposal-sent" rather than "proposal-email" to capture the action orientation.

6. Email Templates and HTML Structure

All branded emails use a consistent HTML template structure designed for maximum compatibility across email clients. The template uses table-based layout (required for Outlook compatibility), inline CSS styles (required for Gmail webmail), and a responsive max-width of 560px for the content area. The design follows the Renewably brand identity with a dark header (#0A0A0A background), yellow accent (#F3D840) for the logo and call-to-action elements, and clean white content areas with subtle shadows and rounded corners.

6.1 Template Anatomy

Section	Background	Content	Notes
Header	#0A0A0A (dark)	"Renewably" logo text in #F3D840, 24px bold	Fixed branding block, no variations
Content	#FFFFFF (white)	Primary message body with tables and text	Max-width 560px, 40px padding
Highlight Card	#FFFDF5 (cream)	Key metrics (amount, date) in prominent display	Yellow-tinted background, used for amounts
CTA Button	#F3D840 (yellow)	Text in #1A1A1A, rounded pill shape	28px font, full-width centered

Footer	#F9FAF9 (light gray)	Brand tagline, contact details, copyright	12px muted text, centered
--------	----------------------	---	---------------------------

Table 9: Email template section breakdown

6.2 Email Client Compatibility

The templates are tested and compatible with the following email clients: Gmail (web and mobile), Outlook 2016+ (Windows and macOS), Apple Mail (macOS and iOS), Thunderbird, Yahoo Mail, and Samsung Email. The table-based layout with inline CSS ensures rendering consistency across these clients. Some newer CSS features like flexbox, CSS Grid, and backdrop-filter are deliberately avoided as they have inconsistent support in email clients, particularly Outlook which uses the Word rendering engine for HTML emails.

7. Error Handling and Retry Logic

The Postmark module implements a two-layer error handling strategy. The first layer handles API-level errors returned by Postmark (HTTP 4xx/5xx responses with error codes), while the second layer catches network-level failures (DNS resolution errors, timeouts, connection refused). Both layers log detailed error information to the console and re-throw the error to the caller, allowing the upstream API route to decide whether to return an error to the client or handle it gracefully.

Currently, the module does not implement automatic retry logic. If a Postmark API call fails, the error propagates immediately to the caller. For the contact form and AI chat endpoints, this means the email failure does not prevent the lead from being saved to the database, but the notification email to the team may be lost. For production deployments, it is recommended to implement a retry queue using a background job system (such as BullMQ with Redis, or Inngest) that automatically retries failed email sends with exponential backoff. This is listed as a priority improvement in the production readiness checklist in Section 14.

7.1 Postmark Error Codes

Error Code	HTTP Status	Meaning	Recommended Action
0	200	Success	No action needed
401	401	Invalid API token	Verify POSTMARK_SERVER_TOKEN value
403	403	Inactive sender signature	Verify hello@renewably.ie is confirmed in Postmark
406	406	Inactive API token	Check token status in Postmark dashboard
422	422	Invalid request body	Check parameter types and required fields
429	429	Rate limit exceeded	Implement backoff, check sending volume

500	500	Postmark server error	Retry with exponential backoff
-----	-----	-----------------------	--------------------------------

Table 10: Common Postmark API error codes

8. Webhook Configuration

Postmark supports inbound webhooks that deliver real-time delivery and engagement events to your application. While the current implementation does not include a dedicated webhook endpoint for Postmark events, the Supabase schema (supabase-schema.sql) includes an email_log table designed to store delivery status, bounce information, and engagement data received from Postmark webhooks. Setting up the webhook is a recommended production step that enables the CRM to track email delivery, automatically mark emails as bounced, and trigger follow-up actions based on customer engagement signals.

8.1 Recommended Webhook Events

Event	Description	CRM Integration Use
Delivery	Email was accepted by the recipient server	Update email_log status to "delivered"
Bounce	Email could not be delivered (hard or soft)	Mark contact email as invalid, notify sales rep
SpamComplaint	Recipient marked the email as spam	Flag the sending pattern, review email content
Open	Recipient opened the email	Update email_log.opened_at, trigger follow-up workflow
Click	Recipient clicked a link in the email	Log the clicked URL, update lead engagement score
SubscriptionChange	Recipient unsubscribed	Update contact preference, suppress future emails

Table 11: Recommended Postmark webhook events

8.2 Webhook Endpoint (Recommended Implementation)

A webhook endpoint should be created at POST /api/webhooks/postmark. This endpoint must be excluded from authentication middleware (similar to the Stripe billing webhook). The endpoint should parse the Postmark webhook payload, validate the event structure, and upsert records into the email_log table in Supabase. The Postmark webhook payload includes a MessageID field that maps directly to the PostmarkResponse.MessageID returned when sending emails, enabling correlation between sent messages and delivery events.

9. Environment Variables

The Postmark integration depends on two environment variables for configuration. Both are optional during development (the module degrades gracefully) but required for production email delivery. These variables should be stored in the hosting platform environment configuration, never in source code or version control.

Variable	Required	Default	Description
POSTMARK_SERVER_TOKEN	Production	(empty)	Postmark server API token from the Postmark dashboard
FROM_EMAIL	No	hello@renewably.ie	Sender email address. Must be verified in Postmark.

Table 12: Postmark environment variables

10. Testing and Debugging

Postmark provides a sandbox mode for testing email delivery without sending real emails. To use sandbox mode, configure the POSTMARK_SERVER_TOKEN with a sandbox token from the Postmark dashboard. In sandbox mode, all emails are accepted by the API but not actually delivered, allowing you to verify API integration, template rendering, and webhook handling without spamming real recipients. Alternatively, if no token is configured, the module returns a synthetic success response with a console warning, which is suitable for local development and unit testing scenarios.

For debugging email rendering, use the Postmark template preview feature in the dashboard to test how HTML emails render across different email clients. The Postmark API also returns a MessageID for every sent email, which can be used to look up the exact delivery status, open/click events, and full message content in the Postmark activity feed. When debugging email-related issues in the CRM, search the server logs for the "[Postmark]" prefix to find all email-related log entries including success confirmations, API errors, and token warnings.

10.1 Local Testing Checklist

Step	Action	Expected Result
1	Submit the public contact form at /contact	Console shows "[Postmark] Skipping request" warning
2	Set POSTMARK_SERVER_TOKEN to a sandbox token	API returns success with MessageID
3	Submit the contact form again	Email appears in Postmark sandbox activity feed
4	Create a proposal in CRM and click "Send"	Proposal email rendered with correct data
5	Create an invoice and click "Send"	Invoice email shows correct EUR formatting
6	Check Postmark activity feed	All emails show correct tags and recipients

Table 13: Local testing checklist for Postmark integration

11. Rate Limits and Quotas

Postmark enforces API rate limits to protect the service from abuse. The standard Postmark plan allows up to 100 API requests per second. Each individual email send counts as one API request. If the rate limit is exceeded, the API returns a 429 Too Many Requests response with a Retry-After header indicating how long to wait before retrying. The current implementation does not implement client-side rate limiting for outgoing emails, relying on the CRM workflow automation patterns (which typically send one email per user action) to stay within limits naturally.

For bulk email scenarios (e.g., sending newsletters or mass notifications to all installer contacts), the implementation should add a rate limiter that batches emails and respects the 100 req/sec ceiling. A simple approach is to use a queue with a 10ms delay between sends (100 emails per second). More sophisticated implementations could use the Postmark Batch Send API endpoint, which accepts up to 500 messages in a single API call, significantly reducing the rate limit impact for bulk operations.

12. CRM Module Integration Map

The Postmark integration connects to multiple CRM modules. The following table provides a complete map of every integration point, showing which API route triggers email sending, which function is called, what tag is assigned, and who receives the email. This map is essential for debugging email delivery issues and understanding the full email flow within the CRM platform.

CRM Module	API Route	Function Called	Tag	Recipient
Contact Form	POST /api/contact	sendContactNotification()	contact-form	hello@renewably.ie
Contact Form	POST /api/contact	sendWelcomeEmail()	welcome-auto-reply	Form submitter
AI Chatbot	POST /api/chat	sendContactNotification()	chat-lead-capture	hello@renewably.ie
Proposals	POST /api/crm/proposals/[id]/send	sendProposalEmail()	proposal-sent	Customer
Invoices	POST /api/crm/invoices/[id]/send	sendInvoiceEmail()	invoice-sent	Customer

Table 14: Complete CRM-to-Postmark integration map

13. Database Integration (email_log Table)

The Supabase production schema includes an email_log table designed to store a record of every email sent through the Postmark integration. This table captures the MessageID returned by Postmark, the recipient, subject, tag, and delivery status. When the Postmark webhook endpoint is implemented (Section 8),

delivery events (bounces, opens, clicks) will update corresponding records in this table, providing a complete audit trail of all email communication within the CRM.

Column	Type	Description
id	UUID (PK)	Unique identifier for the log record
message_id	TEXT	Postmark MessageID for webhook correlation
postmark_tag	TEXT	Tag assigned when sending the email
recipient	TEXT	Recipient email address
subject	TEXT	Email subject line
status	email_status (enum)	sent / delivered / bounced / opened / failed
opened_at	TIMESTAMPTZ	When the email was first opened (from webhook)
clicked_at	TIMESTAMPTZ	When a link was first clicked (from webhook)
bounce_reason	TEXT	Bounce description if the email bounced
crm_entity_type	TEXT	Polymorphic: contact / deal / proposal / invoice
crm_entity_id	UUID	FK to the related CRM entity
created_at	TIMESTAMPTZ	When the email was sent
metadata	JSONB	Additional data (template ID, error codes, etc.)

Table 15: email_log table schema

14. Migration from Dev to Production

Moving the Postmark integration from development to production requires several configuration steps. The development environment works without a Postmark token (emails are silently skipped), but production requires a fully configured Postmark server with verified sender domains, DKIM records, and SPF/DKIM/DMARC authentication to ensure maximum deliverability. Follow the checklist below to ensure a smooth production deployment of the email system.

14.1 Production Readiness Checklist

#	Task	Priority	Status
1	Create a Postmark account and server at postmarkapp.com	Critical	Pending
2	Verify the sending domain: renewably.ie (add DNS records for SPF, DKIM, DMARC)	Critical	Pending
3	Verify the sender signature: hello@renewably.ie	Critical	Pending

4	Generate a server API token and set POSTMARK_SERVER_TOKEN in production env	Critical	Pending
5	Set FROM_EMAIL to hello@renewably.ie in production env	High	Pending
6	Configure Postmark webhook endpoint at POST /api/webhooks/postmark	High	Pending
7	Enable delivery, bounce, spam complaint, open, and click webhook events	High	Pending
8	Test end-to-end: contact form, welcome email, proposal, invoice	Critical	Pending
9	Verify email rendering in Gmail, Outlook, Apple Mail using Postmark preview	Medium	Pending
10	Implement retry queue for failed email sends (BullMQ/Inngest recommended)	High	Pending
11	Set up Postmark alerting for bounce rate spikes and delivery failures	Medium	Pending
12	Add email_log insertion after every successful send in production	Medium	Pending

Table 16: Postmark production readiness checklist

15. Quick Reference Card

This section provides a condensed quick reference for the most common Postmark integration tasks. Keep this page bookmarked for rapid lookups during development.

Import

```
import { sendEmail, sendTemplate, sendContactNotification, sendWelcomeEmail,
sendProposalEmail, sendInvoiceEmail } from "@lib/postmark";
```

API Endpoints

Endpoint	Method	Purpose
https://api.postmarkapp.com/email/withTemplate	POST	Send raw HTML email
https://api.postmarkapp.com/email/send	POST	Send template-based email
https://api.postmarkapp.com/email/batch	POST	Send up to 500 emails in one request
https://api.postmarkapp.com/messages/outline/{id}	GET	Get message delivery status
https://api.postmarkapp.com/stats/overview	GET	Get sending statistics

Table 17: Postmark REST API endpoints

HTTP Headers (Every Request)

```
Accept: application/json<br/>Content-Type:  
application/json<br/>X-Postmark-Server-Token: YOUR_TOKEN
```

Tag Quick Reference

```
contact-form -> New enquiry notification to team<br/>welcome-auto-reply -> Auto-reply  
to form submitter<br/>proposal-sent -> Proposal delivered to customer<br/>invoice-sent  
-> Invoice delivered to customer<br/>chat-lead-capture -> AI chat lead capture  
notification
```

Error Response Format

```
{<br/> "ErrorCode": 422,<br/> "Message": "Invalid 'To' address: not-an-email",<br/>  
"MessageID": "",<br/> "SubmittedAt": "",<br/> "To": "not-an-email"<br/>}
```